

Lecture 11

Overfitting and regularization

COURSE MATH 637: Mathematical Techniques in Data Science

WEBSITE <https://jacobcalvert.com/teaching/math637fall2026/>

INSTRUCTOR Jacob Calvert (calvert@udel.edu)

OFFICE HOURS MW 2:45pm–3:45pm, Ewing Hall 509

Announcements & Reminders

- Homework 2 is **due Monday before class**
(I forgot to update this slide last time!)

Plan for today

- | | |
|--|--------|
| • Training and testing sets | 5 min |
| • Example: predicting sales from advertising | 7 min |
| • Overfitting | 7 min |
| • Ridge regression | 11 min |
| • Choosing λ : cross-validation | 10 min |
| • Lasso and elastic net | 8 min |

Definition: Generalization error

- So far, we've only discussed minimizing MSE on a dataset.
- But what we *actually* care about is predicting well on new data!
- We care about **generalization error**—the MSE on *unseen* data.

Definition: Generalization error

- So far, we've only discussed minimizing MSE on a dataset.
- But what we *actually* care about is predicting well on new data!
- We care about **generalization error**—the MSE on *unseen* data.

Definition

The **generalization error** associated with parameter vector $\hat{\theta}$ is

$$\text{Err}(\hat{\theta}) = \mathbb{E}[(y_{\text{new}} - \hat{\theta}^T x_{\text{new}})^2],$$

where $(x_{\text{new}}, y_{\text{new}})$ is a new example, independent of the data used to estimate $\hat{\theta}$, but from the same distribution.

Definition: Generalization error

- So far, we've only discussed minimizing MSE on a dataset.
- But what we *actually* care about is predicting well on new data!
- We care about **generalization error**—the MSE on *unseen* data.

Definition

The **generalization error** associated with parameter vector $\hat{\theta}$ is

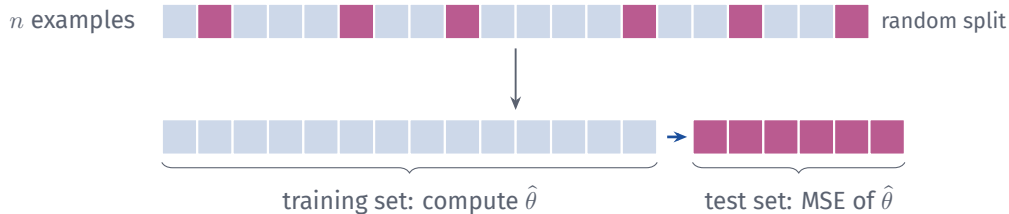
$$\text{Err}(\hat{\theta}) = \mathbb{E}[(y_{\text{new}} - \hat{\theta}^T x_{\text{new}})^2],$$

where $(x_{\text{new}}, y_{\text{new}})$ is a new example, independent of the data used to estimate $\hat{\theta}$, but from the same distribution.

Problem

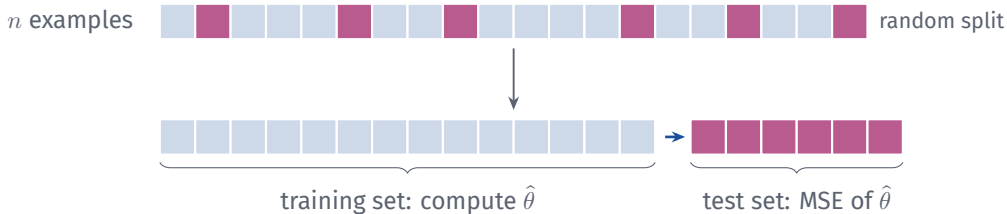
We don't know the distribution of $(x_{\text{new}}, y_{\text{new}})$, so we can't calculate $\text{Err}(\hat{\theta})$.

Idea: Estimate $\text{Err}(\hat{\theta})$ using a **test set**



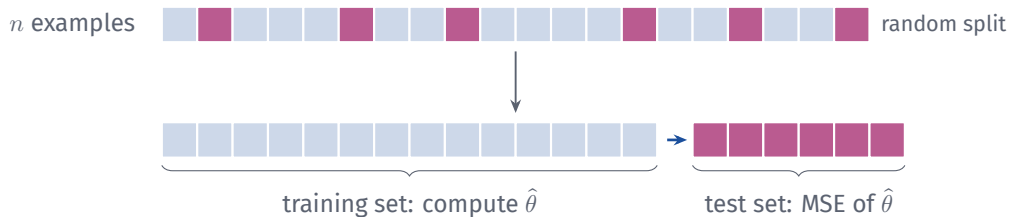
1. Split at random, **before** any preprocessing or fitting.

Idea: Estimate $\text{Err}(\hat{\theta})$ using a **test set**



1. Split at random, **before** any preprocessing or fitting.
2. Fit $\hat{\theta}$ as we've discussed, using only the **training set**.

Idea: Estimate $\text{Err}(\hat{\theta})$ using a **test set**



1. Split at random, **before** any preprocessing or fitting.
2. Fit $\hat{\theta}$ as we've discussed, using only the **training set**.
3. Estimate the generalization error by the MSE of $\hat{\theta}$ on the **test set**:

$$\frac{1}{n_{\text{test}}} \sum_{i \in \text{test}} (y_i - \hat{\theta}^T x_i)^2.$$

Plan for today

- Training and testing sets 5 min
- Example: predicting sales from advertising 7 min
- Overfitting 7 min
- Ridge regression 11 min
- Choosing λ : cross-validation 10 min
- Lasso and elastic net 8 min

Example: Advertising data

For each of $n = 200$ markets:

- $x_i \in \mathbb{R}^3$: the budgets for TV, radio and newspaper advertising of a product, in thousands of USD
- y_i : sales of the product, in thousands of units

market	TV	radio	news.	sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

James, Witten, Hastie and Tibshirani, *An Introduction to Statistical Learning* (2021)

Example: Advertising data

For each of $n = 200$ markets:

- $x_i \in \mathbb{R}^3$: the budgets for TV, radio and newspaper advertising of a product, in thousands of USD
- y_i : sales of the product, in thousands of units

market	TV	radio	news.	sales
1	230.1	37.8	69.2	22.1
2	44.5	39.3	45.1	10.4
3	17.2	45.9	69.3	9.3
4	151.5	41.3	58.5	18.5
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Question

How well can we predict the sales in a market from the ad budgets?

James, Witten, Hastie and Tibshirani, *An Introduction to Statistical Learning* (2021)

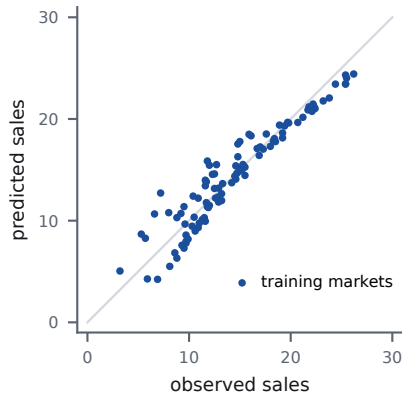
Example: Advertising data

Using 100 of the examples, fit

$$\hat{y} = \theta_0 + \theta_1 x_{\text{TV}} + \theta_2 x_{\text{radio}} + \theta_3 x_{\text{news}}.$$

After standardizing each budget within the training data, least squares gives:

- $\hat{\theta}_1 = 3.90, \hat{\theta}_2 = 2.89, \hat{\theta}_3 = 0.07$



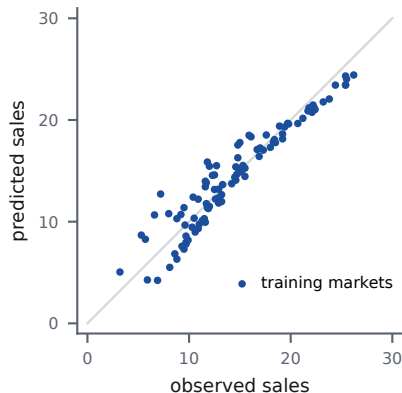
Example: Advertising data

Using 100 of the examples, fit

$$\hat{y} = \theta_0 + \theta_1 x_{\text{TV}} + \theta_2 x_{\text{radio}} + \theta_3 x_{\text{news}}.$$

After standardizing each budget within the training data, least squares gives:

- $\hat{\theta}_1 = 3.90, \hat{\theta}_2 = 2.89, \hat{\theta}_3 = 0.07$
- $\text{MSE}(\hat{\theta}) = 2.78$, so a typical error is $\sqrt{2.78} \approx 1.7$ **thousand units**



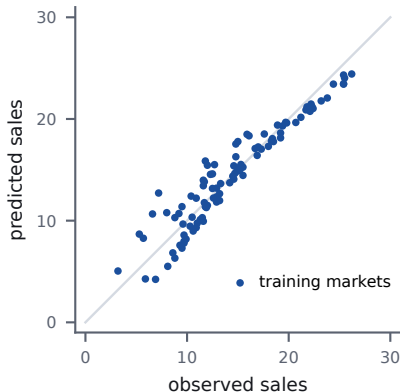
Example: Advertising data

Using 100 of the examples, fit

$$\hat{y} = \theta_0 + \theta_1 x_{\text{TV}} + \theta_2 x_{\text{radio}} + \theta_3 x_{\text{news}}.$$

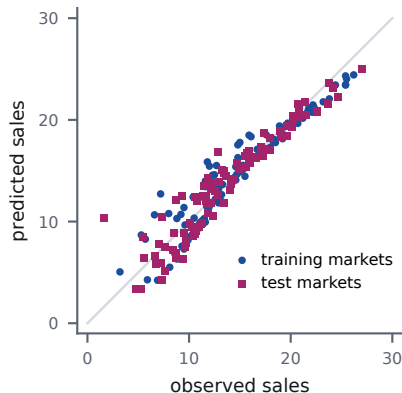
After standardizing each budget within the training data, least squares gives:

- $\hat{\theta}_1 = 3.90, \hat{\theta}_2 = 2.89, \hat{\theta}_3 = 0.07$
- $\text{MSE}(\hat{\theta}) = 2.78$, so a typical error is $\sqrt{2.78} \approx 1.7$ **thousand units**
- **constant model:** $\text{MSE} = 27.3$, so $R^2 \approx 0.9$.

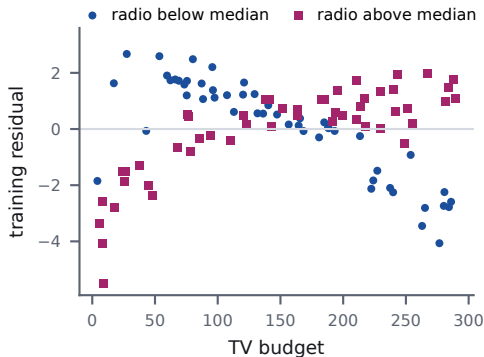


Example: The 100 held-out markets

- The other 100 markets were set aside before fitting
- Test MSE 2.83, compared to training MSE 2.78
- Test MSE is random: over 100 random train/test splits it ranged from 1.93 to 4.37

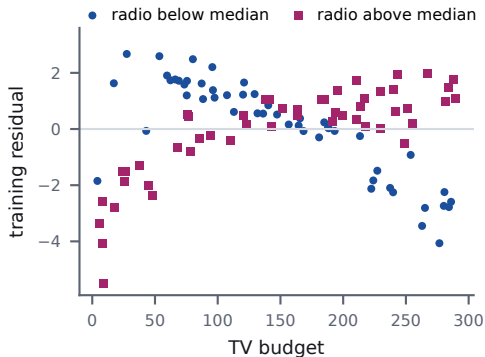


Example: The training residuals have a pattern



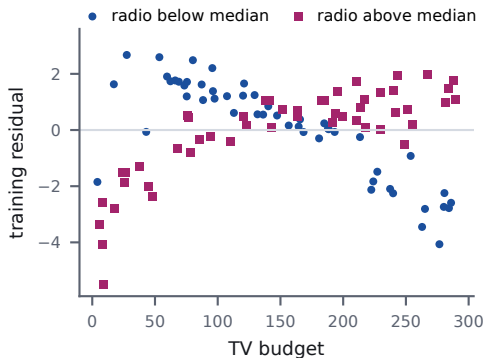
- TV above 150: mean residual $+0.88$ when radio is above its median, -1.35 when below

Example: The training residuals have a pattern



- TV above 150: mean residual $+0.88$ when radio is above its median, -1.35 when below
- Note the large negative residuals at low TV (-5.5 at market 6)

Example: The training residuals have a pattern



- TV above 150: mean residual $+0.88$ when radio is above its median, -1.35 when below
- Note the large negative residuals at low TV (-5.5 at market 6)
- Suggests a TV-radio interaction, which an additive model cannot represent

Example: Higher-order features

Recall that least squares only needs to be linear in θ , so add all higher-order features of degree ≤ 6 :

$$x_{\text{TV}}, x_{\text{TV}} x_{\text{radio}}, x_{\text{TV}}^2, \dots, x_{\text{TV}}^2 x_{\text{radio}} x_{\text{news}}^3, \dots, x_{\text{news}}^6$$

degree	1	2	3	4	5	6	total
features	3	6	10	15	21	28	83

Standardize all columns.

Example: Higher-order features

Recall that least squares only needs to be linear in θ , so add all higher-order features of degree ≤ 6 :

$$x_{\text{TV}}, x_{\text{TV}} x_{\text{radio}}, x_{\text{TV}}^2, \dots, x_{\text{TV}}^2 x_{\text{radio}} x_{\text{news}}^3, \dots, x_{\text{news}}^6$$

degree	1	2	3	4	5	6	total
features	3	6	10	15	21	28	83

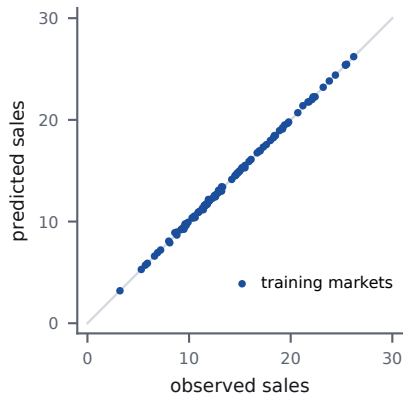
Standardize all columns.

Check

With 84 coefficients and 100 training examples, what do you expect the training MSE to be?

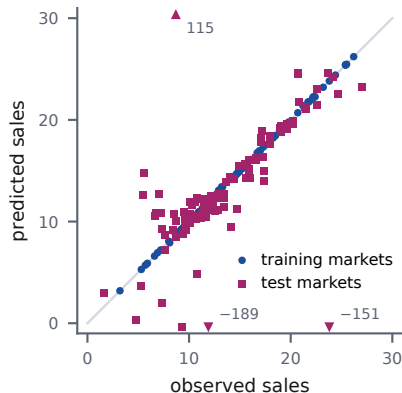
Example: Least squares on 83 features

- Training MSE 0.008



Example: Least squares on 83 features

- Training MSE 0.008
- Test MSE 828
- Most test markets are predicted well, but three have *catastrophic* predictions



Plan for today

- Training and testing sets 5 min
- Example: predicting sales from advertising 7 min
- **Overfitting** 7 min
- Ridge regression 11 min
- Choosing λ : cross-validation 10 min
- Lasso and elastic net 8 min

Example: More features, **better** fit?

	3 budgets	83 features
training MSE	2.78	0.008
test MSE	2.83	828
test MSE, median over 100 splits	2.99	1195
largest $ \hat{\theta}_j $	3.90	126

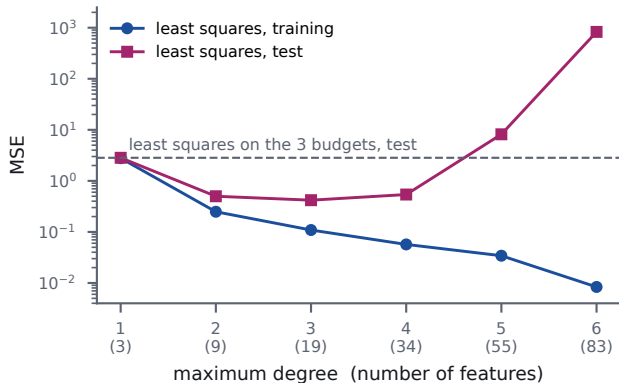
- 83 features did worse than 3 on the test markets in all 100 splits.

Example: More features, **better** fit?

	3 budgets	83 features
training MSE	2.78	0.008
test MSE	2.83	828
test MSE, median over 100 splits	2.99	1195
largest $ \hat{\theta}_j $	3.90	126

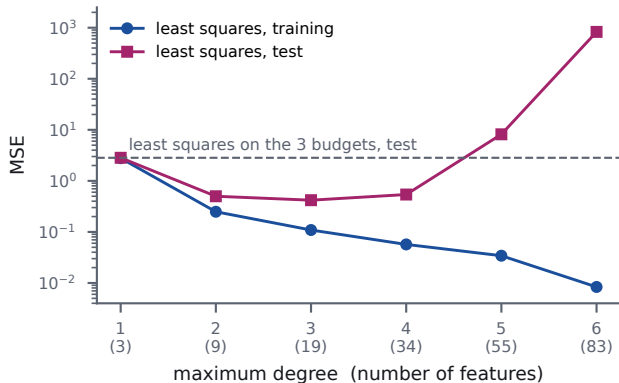
- 83 features did worse than 3 on the test markets in all 100 splits.
- The fit to the training markets improved, and the predictions for new markets got much worse.

Example: Degree by degree



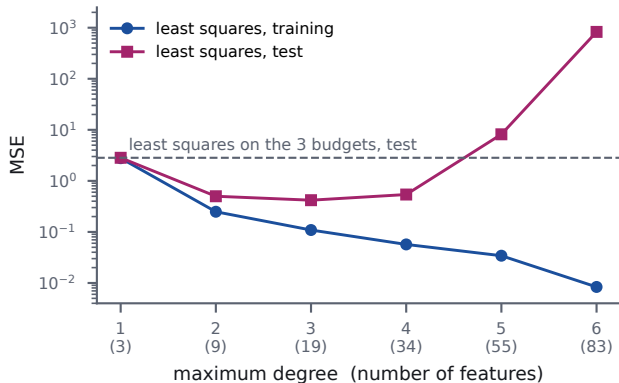
- Degree 2 contains x_{TV} x_{radio} : test MSE 0.50

Example: Degree by degree



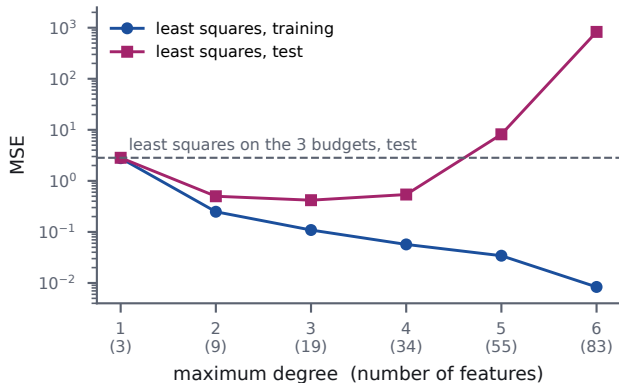
- Degree 2 contains x_{TV} x_{radio} : test MSE 0.50
- Degree 3: 0.42, the smallest MSE on the test set

Example: Degree by degree



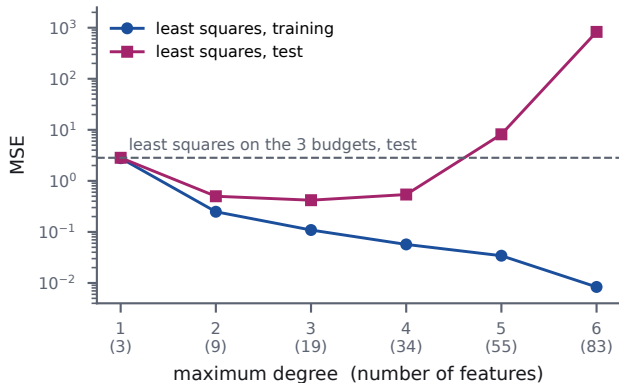
- Degree 2 contains $x_{TV} x_{radio}$: test MSE 0.50
- Degree 3: 0.42, the smallest MSE on the test set
- From degree 4 on, the training MSE keeps falling and the test MSE rises

Example: Degree by degree



- Degree 2 contains $x_{TV} x_{radio}$: test MSE 0.50
- Degree 3: 0.42, the smallest MSE on the test set
- From degree 4 on, the training MSE keeps falling and the test MSE rises

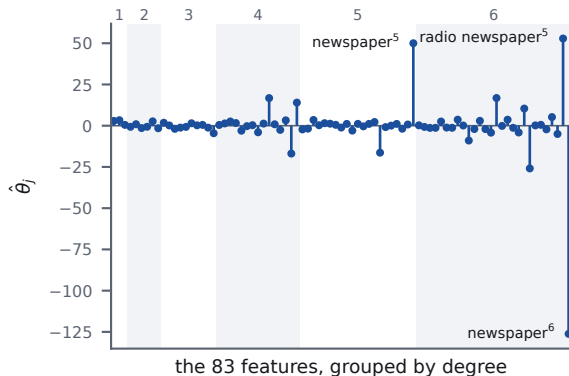
Example: Degree by degree



- Degree 2 contains x_{TV} x_{radio} : test MSE 0.50
- Degree 3: 0.42, the smallest MSE on the test set
- From degree 4 on, the training MSE keeps falling and the test MSE rises

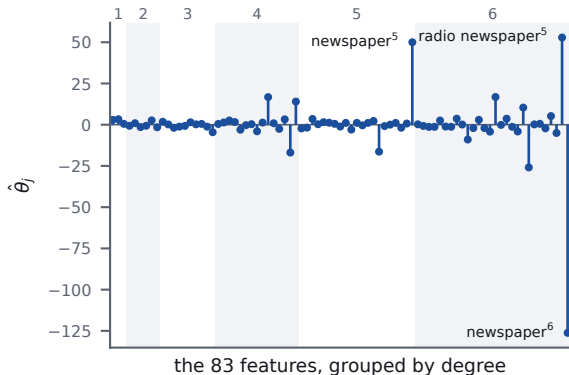
Watch out: Choosing degree this way would be poor methodology.

Example: The problem is **huge** coefficients



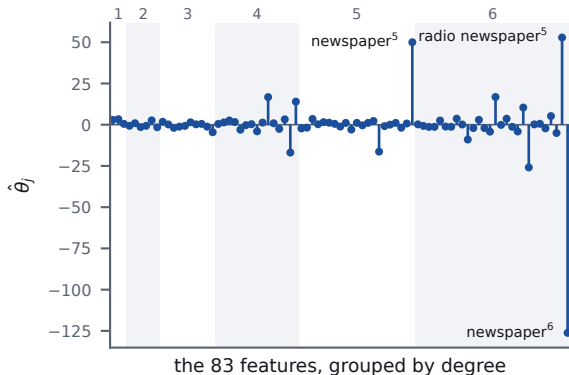
- Example 166 has $TV = 234.5$, $radio = 3.4$, $newspaper = 84.8$, and sales of 11.9

Example: The problem is **huge** coefficients



- Example 166 has $TV = 234.5$, $radio = 3.4$, $newspaper = 84.8$, and sales of 11.9
- Predictions: 14.3 (3 budgets), -189 (83 features)

Example: The problem is huge coefficients



- Example 166 has TV = 234.5, radio = 3.4, newspaper = 84.8, and sales of 11.9
- Predictions: 14.3 (3 budgets), -189 (83 features)
- Having a newspaper budget of \$1000 higher moves predicted sales by 0.003 (3 budgets) versus **-21** (83 features)

Definition: Overfitting

Definition (Overfitting)

A fitted model **overfits** when it fits the training data much better than it predicts new data: small training MSE, large test MSE.

- the 83-feature fit puts large coefficients of opposite signs on similar features ($x_{\text{news}}^6: -126$; $x_{\text{news}}^5: 50$; $x_{\text{radio}} x_{\text{news}}^5: 53$), which nearly cancel on the training markets and not on new ones

Definition: Overfitting

Definition (Overfitting)

A fitted model **overfits** when it fits the training data much better than it predicts new data: small training MSE, large test MSE.

- the 83-feature fit puts large coefficients of opposite signs on similar features ($x_{\text{news}}^6: -126$; $x_{\text{news}}^5: 50$; $x_{\text{radio}} x_{\text{news}}^5: 53$), which nearly cancel on the training markets and not on new ones

Definition: Overfitting

Definition (Overfitting)

A fitted model **overfits** when it fits the training data much better than it predicts new data: small training MSE, large test MSE.

- the 83-feature fit puts large coefficients of opposite signs on similar features ($x_{\text{news}}^6: -126$; $x_{\text{news}}^5: 50$; $x_{\text{radio}} x_{\text{news}}^5: 53$), which nearly cancel on the training markets and not on new ones

Idea: Penalize large coefficients. Choose θ to make $\text{MSE}(\theta)$ small **and** $\|\theta\|$ small.

Plan for today

- Training and testing sets 5 min
- Example: predicting sales from advertising 7 min
- Overfitting 7 min
- Ridge regression 11 min
- Choosing λ : cross-validation 10 min
- Lasso and elastic net 8 min

Definition: Ridge regression

Definition (Ridge regression)

Let $X \in \mathbb{R}^{n \times p}$ be a **standardized data** matrix and $y \in \mathbb{R}^n$ be centered. For $\lambda > 0$, find $\theta \in \mathbb{R}^p$ minimizing

$$\text{MSE}_{\lambda}(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2 = \frac{1}{n} \|y - X\theta\|^2 + \lambda \|\theta\|^2.$$

Definition: Ridge regression

Definition (Ridge regression)

Let $X \in \mathbb{R}^{n \times p}$ be a **standardized data** matrix and $y \in \mathbb{R}^n$ be centered. For $\lambda > 0$, find $\theta \in \mathbb{R}^p$ minimizing

$$\text{MSE}_{\lambda}(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2 = \frac{1}{n} \|y - X\theta\|^2 + \lambda \|\theta\|^2.$$

Note: By convention, X has standardized columns and no column of ones. We fit to $y - \bar{y}$, and the intercept is $\bar{y}$, so it is not “pulled” toward 0.

Definition: Ridge regression

Definition (Ridge regression)

Let $X \in \mathbb{R}^{n \times p}$ be a **standardized data** matrix and $y \in \mathbb{R}^n$ be centered. For $\lambda > 0$, find $\theta \in \mathbb{R}^p$ minimizing

$$\text{MSE}_\lambda(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2 = \frac{1}{n} \|y - X\theta\|^2 + \lambda \|\theta\|^2.$$

Note: By convention, X has standardized columns and no column of ones. We fit to $y - \bar{y}$, and the intercept is $\bar{y}$, so it is not “pulled” toward 0.

Idea: Penalize large coefficients θ_i through $\lambda \|\theta\|^2$ to prevent overfitting!

Aside: Ridge regression was invented at UD

TECHNOMETRICS

VOL. 12, No. 1

FEBRUARY 1970

Ridge Regression: Biased Estimation for Nonorthogonal Problems

ARTHUR E. HOERL AND ROBERT W. KENNARD

University of Delaware and E. I. du Pont de Nemours & Co.

In multiple regression it is shown that parameter estimates based on minimum residual sum of squares have a high probability of being unsatisfactory, if not incorrect, if the prediction vectors are not orthogonal. Proposed is an estimation procedure based on adding small positive quantities to the diagonal of $\mathbf{X}'\mathbf{X}$. Introduced is the ridge trace, a method for showing in two dimensions the effects of nonorthogonality. It is then shown how to augment $\mathbf{X}'\mathbf{X}$ to obtain biased estimates with smaller mean square error.

Ridge regression always has **one** solution

- By definition:

$$\text{MSE}_\lambda(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2$$

Ridge regression always has **one** solution

- By definition:

$$\text{MSE}_\lambda(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2$$

- Using gradient rules from L9:

$$\nabla \text{MSE}_\lambda(\theta) = \frac{2}{n} X^T (X\theta - y) + 2\lambda\theta$$

Ridge regression always has **one** solution

- By definition:

$$\text{MSE}_\lambda(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2$$

- Using gradient rules from L9:

$$\nabla \text{MSE}_\lambda(\theta) = \frac{2}{n} X^\top (X\theta - y) + 2\lambda\theta$$

- By setting gradient to zero:

$$(X^\top X + n\lambda I)\hat{\theta}_\lambda = X^\top y$$

Ridge regression always has **one** solution

- By definition:
- Using gradient rules from L9:
- By setting gradient to zero:
- Best parameter vector is:

$$\text{MSE}_\lambda(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2$$

$$\nabla \text{MSE}_\lambda(\theta) = \frac{2}{n} X^\top (X\theta - y) + 2\lambda\theta$$

$$(X^\top X + n\lambda I)\hat{\theta}_\lambda = X^\top y$$

$$\hat{\theta}_\lambda = (X^\top X + n\lambda I)^{-1} X^\top y$$

Ridge regression always has **one** solution

- By definition:
- Using gradient rules from L9:
- By setting gradient to zero:
- Best parameter vector is:

$$\text{MSE}_\lambda(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2$$

$$\nabla \text{MSE}_\lambda(\theta) = \frac{2}{n} X^\top (X\theta - y) + 2\lambda\theta$$

$$(X^\top X + n\lambda I)\hat{\theta}_\lambda = X^\top y$$

$$\hat{\theta}_\lambda = (X^\top X + n\lambda I)^{-1} X^\top y$$

Ridge regression always has **one** solution

- By definition: $\text{MSE}_\lambda(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2$
- Using gradient rules from L9: $\nabla \text{MSE}_\lambda(\theta) = \frac{2}{n} X^\top (X\theta - y) + 2\lambda\theta$
- By setting gradient to zero: $(X^\top X + n\lambda I)\hat{\theta}_\lambda = X^\top y$
- Best parameter vector is: $\hat{\theta}_\lambda = (X^\top X + n\lambda I)^{-1} X^\top y$

Idea: $X^\top X + n\lambda I$ is *always* invertible! For every $\lambda > 0$ and every $v \neq 0$,

$$v^\top (X^\top X + n\lambda I)v = \|Xv\|^2 + n\lambda \|v\|^2 > 0.$$

Ridge regression always has **one** solution

- By definition: $\text{MSE}_\lambda(\theta) = \text{MSE}(\theta) + \lambda \|\theta\|^2$
- Using gradient rules from L9: $\nabla \text{MSE}_\lambda(\theta) = \frac{2}{n} X^\top (X\theta - y) + 2\lambda\theta$
- By setting gradient to zero: $(X^\top X + n\lambda I)\hat{\theta}_\lambda = X^\top y$
- Best parameter vector is: $\hat{\theta}_\lambda = (X^\top X + n\lambda I)^{-1} X^\top y$

Idea: $X^\top X + n\lambda I$ is *always* invertible! For every $\lambda > 0$ and every $v \neq 0$,

$$v^\top (X^\top X + n\lambda I)v = \|Xv\|^2 + n\lambda \|v\|^2 > 0.$$

Note: $\hat{\theta}_\lambda \rightarrow X^+ y$ as $\lambda \downarrow 0$ because the pseudoinverse solution to least squares has the smallest norm!

Python: Least squares vs. ridge

```
X1 = np.column_stack([np.ones(n), Z])    # ones column
theta, *_ = np.linalg.lstsq(X1, y, rcond=None)

scaler = StandardScaler().fit(Z)          # training data
X = scaler.transform(Z)
A = X.T @ X + n * lam * np.eye(X.shape[1])
theta_lam = np.linalg.solve(A, X.T @ (y - y.mean()))
y_new = y.mean() + scaler.transform(Z_new) @ theta_lam
```

- Ridge fits $y - \bar{y}$ with no column of ones. Predictions add back the intercept $\bar{y}$.

Python: Least squares vs. ridge

```
X1 = np.column_stack([np.ones(n), Z])    # ones column
theta, *_ = np.linalg.lstsq(X1, y, rcond=None)

scaler = StandardScaler().fit(Z)          # training data
X = scaler.transform(Z)
A = X.T @ X + n * lam * np.eye(X.shape[1])
theta_lam = np.linalg.solve(A, X.T @ (y - y.mean()))
y_new = y.mean() + scaler.transform(Z_new) @ theta_lam
```

- Ridge fits $y - \bar{y}$ with no column of ones. Predictions add back the intercept $\bar{y}$.
- `scaler` keeps the **training** mean and standard deviation and applies them to new rows.

Python: Ridge in scikit-learn

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import Ridge

def expand(d):
    return [StandardScaler(),
            PolynomialFeatures(d, include_bias=False),
            StandardScaler()]
model = make_pipeline(*expand(6), Ridge(alpha=n * lam))
model.fit(Z_train, y_train)
```

- Ridge minimizes $\|y - X\theta\|^2 + \alpha \|\theta\|^2$, so $\alpha = n\lambda$.

Example: Using ridge on the expanded sales data

	training MSE	test MSE	largest $ \hat{\theta}_j $
$\lambda = 0$ (least squares)	0.008	828	126
$\lambda = 10^{-6}$	0.010	49.9	11.8
$\lambda = 10^{-4}$	0.015	4.02	3.26
$\lambda = 10^{-2}$	0.049	0.651	2.73
$\lambda = 10^{-1}$	0.333	1.04	1.96
$\lambda = 1$	2.34	4.39	0.87
original data , least squares	2.78	2.83	3.90

Example: Using ridge on the expanded sales data

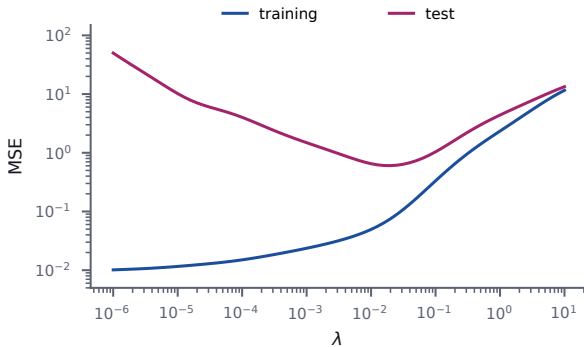
	training MSE	test MSE	largest $ \hat{\theta}_j $
$\lambda = 0$ (least squares)	0.008	828	126
$\lambda = 10^{-6}$	0.010	49.9	11.8
$\lambda = 10^{-4}$	0.015	4.02	3.26
$\lambda = 10^{-2}$	0.049	0.651	2.73
$\lambda = 10^{-1}$	0.333	1.04	1.96
$\lambda = 1$	2.34	4.39	0.87
original data , least squares	2.78	2.83	3.90

- As λ grows, coefficients shrink and training MSE rises (**why?**).
- Test MSE falls from 828 to 0.65, then rises.
- At $\lambda = 10^{-2}$, both MSEs below those of original data with least squares.

Definition: Hyperparameter

Definition

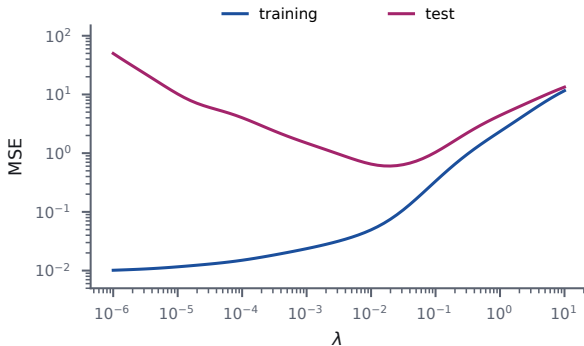
A **hyperparameter**, like λ or the degree of polynomial features we consider, is set *before fitting*, not found by minimizing the training MSE.



Definition: Hyperparameter

Question

The training MSE is smallest at $\lambda = 0$, so we *cannot* use it to choose λ . So how should we choose λ ? The test set MSE? Something else?



Plan for today

- Training and testing sets 5 min
- Example: predicting sales from advertising 7 min
- Overfitting 7 min
- Ridge regression 11 min
- Choosing λ : cross-validation 10 min
- Lasso and elastic net 8 min

Idea: Introduce a validation set



Watch out: If you use the test set to choose anything (e.g., λ , a set of features), the test MSE for the chosen model is optimistic.

Idea: Introduce a validation set



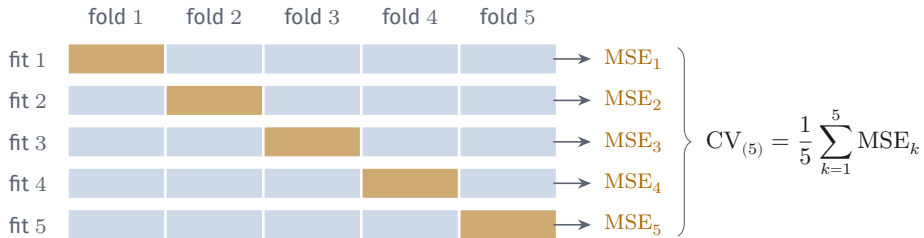
Watch out: If you use the test set to choose anything (e.g., λ , a set of features), the test MSE for the chosen model is optimistic.

Note: Our earlier degree and λ comparisons used the test markets this way; those were just illustrations.

Problem

The validation set leaves less data for training and testing!

Definition: K -fold cross-validation



Definition (K -fold cross-validation)

Partition the training set into K “folds.” For each k , fit without k -th fold and compute MSE_k on fold k . Then $\text{CV}_{(K)} = \frac{1}{K} \sum_{k=1}^K \text{MSE}_k$.

Choosing λ with K -fold cross-validation

- Approach:

Choosing λ with K -fold cross-validation

- Approach:
 1. For each candidate λ , compute $CV_{(K)}$.

Choosing λ with K -fold cross-validation

- Approach:
 1. For each candidate λ , compute $CV_{(K)}$.
 2. Choose the λ that minimizes it.

Choosing λ with K -fold cross-validation

- Approach:
 1. For each candidate λ , compute $CV_{(K)}$.
 2. Choose the λ that minimizes it.
 3. Then, refit the model with this λ using the **entire training set**.

Choosing λ with K -fold cross-validation

- Approach:
 1. For each candidate λ , compute $CV_{(K)}$.
 2. Choose the λ that minimizes it.
 3. Then, refit the model with this λ using the **entire training set**.
- With smaller K , each fit uses fewer examples, so CV tends to be more pessimistic about the final model fit to all n ; $K = 5$ or 10 is usual

Choosing λ with K -fold cross-validation

- Approach:
 1. For each candidate λ , compute $CV_{(K)}$.
 2. Choose the λ that minimizes it.
 3. Then, refit the model with this λ using the **entire training set**.
- With smaller K , each fit uses fewer examples, so CV tends to be more pessimistic about the final model fit to all n ; $K = 5$ or 10 is usual
- The test set is **used only once, at the very end!**

Python: K -fold CV for ridge

```
from sklearn.model_selection import GridSearchCV

pipe = make_pipeline(*expand(6), Ridge())
alphas = np.logspace(-6, 3, 91)
search = GridSearchCV(
    pipe, {"ridge__alpha": alphas}, cv=5,
    scoring="neg_mean_squared_error")
search.fit(Z_train, y_train)
```

- for each α : 5 fits, each refitting the pipeline, scalers included, on four folds and scoring it on the fifth

Python: K -fold CV for ridge

```
from sklearn.model_selection import GridSearchCV

pipe = make_pipeline(*expand(6), Ridge())
alphas = np.logspace(-6, 3, 91)
search = GridSearchCV(
    pipe, {"ridge__alpha": alphas}, cv=5,
    scoring="neg_mean_squared_error")
search.fit(Z_train, y_train)
```

- for each α : 5 fits, each refitting the pipeline, scalers included, on four folds and scoring it on the fifth
- "ridge__alpha" is the `alpha` of the pipeline's `Ridge` step; in our notation $\lambda = \alpha/n$

Python: Using the chosen λ

```
best_alpha = search.best_params_["ridge__alpha"]  
cv_mse = -search.best_score_  
y_pred = search.predict(Z_test)  
test_mse = np.mean((y_test - y_pred) ** 2)
```

- steps 1–3 of the approach: CV for each α , take the minimizer, refit on the **entire training set** (`refit=True`, the default)

Python: Using the chosen λ

```
best_alpha = search.best_params_["ridge__alpha"]  
cv_mse = -search.best_score_  
y_pred = search.predict(Z_test)  
test_mse = np.mean((y_test - y_pred) ** 2)
```

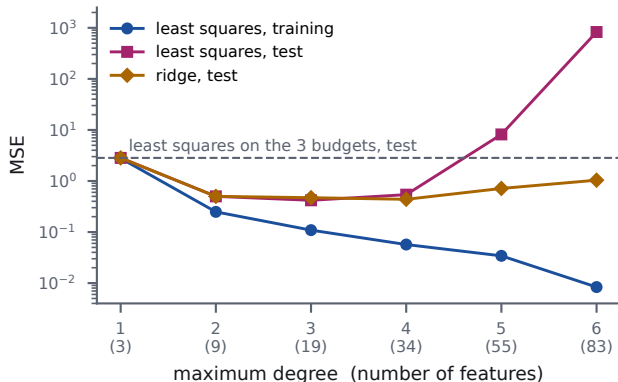
- steps 1–3 of the approach: CV for each α , take the minimizer, refit on the **entire training set** (`refit=True`, the default)
- the test set appears only in the last two lines

Python: Using the chosen λ

```
best_alpha = search.best_params_["ridge__alpha"]  
cv_mse = -search.best_score_  
y_pred = search.predict(Z_test)  
test_mse = np.mean((y_test - y_pred) ** 2)
```

- steps 1–3 of the approach: CV for each α , take the minimizer, refit on the **entire training set** (`refit=True`, the default)
- the test set appears only in the last two lines
- our split: $\alpha = 10$ ($\lambda = 0.1$), CV MSE 1.22, test MSE **1.04**, against 828 for least squares

Ridge at **every** degree

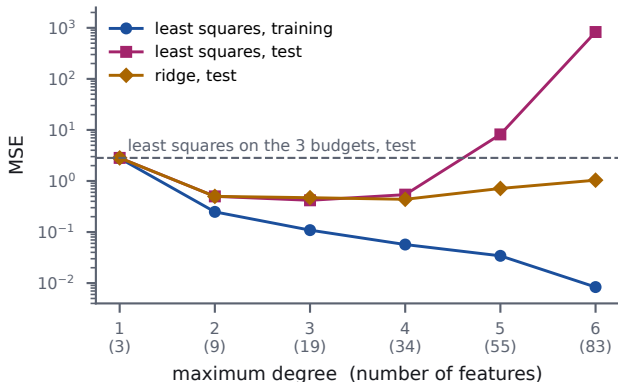


Median test MSE, 100 splits

least squares, 3	2.99
least squares, degree 3	0.40
least squares, 83	1195
ridge, 83	0.58

- λ by 5-fold CV on each split's training markets

Ridge at **every** degree



Median test MSE, 100 splits

least squares, 3 2.99

least squares, degree 3 0.40

least squares, 83 1195

ridge, 83 **0.58**

- λ by 5-fold CV on each split's training markets
- ridge beats least squares on 83 always, on 3 in 88%, and at degree 3 in only 26%

Plan for today

- Training and testing sets 5 min
- Example: predicting sales from advertising 7 min
- Overfitting 7 min
- Ridge regression 11 min
- Choosing λ : cross-validation 10 min
- Lasso and elastic net 8 min

Definition: Lasso and elastic net

Definition (Lasso; Santosa & Symes, 1986 and Tibshirani, 1996)

Minimize $\text{MSE}(\theta) + \lambda \sum_j |\theta_j|$.

Definition (Elastic net; Zou & Hastie, 2005)

Minimize $\text{MSE}(\theta) + \lambda_1 \sum_j |\theta_j| + \lambda_2 \|\theta\|^2$.

- same convention as ridge: standardized columns, centered y

F. Santosa and W. Symes, SIAM J. Sci. Stat. Comput. **7** (1986)
R. Tibshirani, J. R. Stat. Soc. B **58** (1996)
H. Zou and T. Hastie, J. R. Stat. Soc. B **67** (2005)

Definition: Lasso and elastic net

Definition (Lasso; Santosa & Symes, 1986 and Tibshirani, 1996)

Minimize $\text{MSE}(\theta) + \lambda \sum_j |\theta_j|$.

Definition (Elastic net; Zou & Hastie, 2005)

Minimize $\text{MSE}(\theta) + \lambda_1 \sum_j |\theta_j| + \lambda_2 \|\theta\|^2$.

- same convention as ridge: standardized columns, centered y
- $|\theta_j|$ has a corner at 0, so many $\hat{\theta}_j$ are exactly 0: the lasso **selects** features

F. Santosa and W. Symes, SIAM J. Sci. Stat. Comput. **7** (1986)
R. Tibshirani, J. R. Stat. Soc. B **58** (1996)
H. Zou and T. Hastie, J. R. Stat. Soc. B **67** (2005)

Definition: Lasso and elastic net

Definition (Lasso; Santosa & Symes, 1986 and Tibshirani, 1996)

Minimize $\text{MSE}(\theta) + \lambda \sum_j |\theta_j|$.

Definition (Elastic net; Zou & Hastie, 2005)

Minimize $\text{MSE}(\theta) + \lambda_1 \sum_j |\theta_j| + \lambda_2 \|\theta\|^2$.

- same convention as ridge: standardized columns, centered y
- $|\theta_j|$ has a corner at 0, so many $\hat{\theta}_j$ are exactly 0: the lasso **selects** features
- no formula for the minimizer; it is computed iteratively

F. Santosa and W. Symes, SIAM J. Sci. Stat. Comput. **7** (1986)
R. Tibshirani, J. R. Stat. Soc. B **58** (1996)
H. Zou and T. Hastie, J. R. Stat. Soc. B **67** (2005)

Which penalty?

	ridge	elastic net	lasso
coefficients	all shrunk, none exactly 0	some exactly 0	many exactly 0
correlated features	share the weight	tends to keep or drop them together	often keeps one, somewhat arbitrarily
computing	one linear system; K -fold CV	iterative; two hyperparameters	iterative; K -fold CV
suits	many small effects	many correlated features, some irrelevant	few features matter; want to read the model

Python: CV around the pipeline

```
from sklearn.linear_model import Lasso, ElasticNet
from sklearn.model_selection import GridSearchCV

def tuned(model, grid):
    pipe = make_pipeline(*expand(6), model)
    return GridSearchCV(pipe, grid, cv=5,
                        scoring="neg_mean_squared_error")
```

- `GridSearchCV` refits the whole pipeline, scalers included, on each fold's training rows, so no validation fold affects the scaling

Python: CV around the pipeline

```
from sklearn.linear_model import Lasso, ElasticNet
from sklearn.model_selection import GridSearchCV

def tuned(model, grid):
    pipe = make_pipeline(*expand(6), model)
    return GridSearchCV(pipe, grid, cv=5,
                        scoring="neg_mean_squared_error")
```

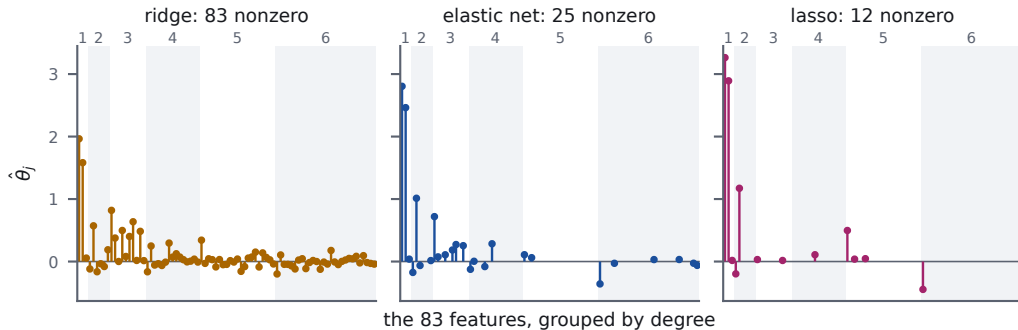
- `GridSearchCV` refits the whole pipeline, scalers included, on each fold's training rows, so no validation fold affects the scaling
- `scoring` makes it choose by MSE

Python: Lasso and elastic net

```
A = np.logspace(-3, 1, 60)
lasso = tuned(Lasso(max_iter=100_000),
              {"lasso__alpha": A})
enet = tuned(ElasticNet(max_iter=100_000),
             {"elasticnet__alpha": A,
              "elasticnet__l1_ratio": [0.5]})
lasso.fit(Z_train, y_train)
```

- `alpha` is $\lambda/2$ for the lasso; for the elastic net, $\lambda_1 = 2\alpha r$, $\lambda_2 = \alpha(1 - r)$,
 $r = \text{`l1\_ratio`}$

Application: Three penalties



Coefficients on the 83 features, each penalty chosen by 5-fold cross-validation on the 100 training markets, with the pipeline refit in each fold.

Application: Three penalties

	features kept	test MSE	median, 100 splits
least squares, 3 budgets	3	2.83	2.99
least squares	83	828	1195
ridge	83	1.04	0.58
elastic net, $r = 0.5$	25	0.37	0.30
lasso	12	0.26	0.24

- the lasso keeps TV, radio and TV·radio in every split

Application: Three penalties

	features kept	test MSE	median, 100 splits
least squares, 3 budgets	3	2.83	2.99
least squares	83	828	1195
ridge	83	1.04	0.58
elastic net, $r = 0.5$	25	0.37	0.30
lasso	12	0.26	0.24

- the lasso keeps TV, radio and TV·radio in every split
- allowed $r \in \{0.1, 0.5, 0.9, 1\}$, CV chooses $r = 1$ (the lasso) in nearly every split

Application: Three penalties

	features kept	test MSE	median, 100 splits
least squares, 3 budgets	3	2.83	2.99
least squares	83	828	1195
ridge	83	1.04	0.58
elastic net, $r = 0.5$	25	0.37	0.30
lasso	12	0.26	0.24

- the lasso keeps TV, radio and TV·radio in every split
- allowed $r \in \{0.1, 0.5, 0.9, 1\}$, CV chooses $r = 1$ (the lasso) in nearly every split

Application: Three penalties

	features kept	test MSE	median, 100 splits
least squares, 3 budgets	3	2.83	2.99
least squares	83	828	1195
ridge	83	1.04	0.58
elastic net, $r = 0.5$	25	0.37	0.30
lasso	12	0.26	0.24

- the lasso keeps TV, radio and TV·radio in every split
- allowed $r \in \{0.1, 0.5, 0.9, 1\}$, CV chooses $r = 1$ (the lasso) in nearly every split

Watch out: The kept features are not “the true model”: they change from split to split.

Bonus: Ridge through the SVD

With $X = \sum_j \sigma_j u_j v_j^\top$ and $\text{rank}(X) = p$, (Check this.)

$$\hat{\theta} = \sum_{j=1}^p \frac{1}{\sigma_j} \langle u_j, y \rangle v_j, \quad \hat{\theta}_\lambda = \sum_{j=1}^p \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} \frac{1}{\sigma_j} \langle u_j, y \rangle v_j.$$

- the factor is near 1 when $\sigma_j^2 \gg n\lambda$ and near 0 when $\sigma_j^2 \ll n\lambda$

Bonus: Ridge through the SVD

With $X = \sum_j \sigma_j u_j v_j^\top$ and $\text{rank}(X) = p$, (Check this.)

$$\hat{\theta} = \sum_{j=1}^p \frac{1}{\sigma_j} \langle u_j, y \rangle v_j, \quad \hat{\theta}_\lambda = \sum_{j=1}^p \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} \frac{1}{\sigma_j} \langle u_j, y \rangle v_j.$$

- the factor is near 1 when $\sigma_j^2 \gg n\lambda$ and near 0 when $\sigma_j^2 \ll n\lambda$
- least squares multiplies by $1/\sigma_j$; for the 83 advertising features the smallest σ_j is 0.0029, so $1/\sigma_j \approx 344$

Bonus: Ridge through the SVD

With $X = \sum_j \sigma_j u_j v_j^\top$ and $\text{rank}(X) = p$, (Check this.)

$$\hat{\theta} = \sum_{j=1}^p \frac{1}{\sigma_j} \langle u_j, y \rangle v_j, \quad \hat{\theta}_\lambda = \sum_{j=1}^p \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} \frac{1}{\sigma_j} \langle u_j, y \rangle v_j.$$

- the factor is near 1 when $\sigma_j^2 \gg n\lambda$ and near 0 when $\sigma_j^2 \ll n\lambda$
- least squares multiplies by $1/\sigma_j$; for the 83 advertising features the smallest σ_j is 0.0029, so $1/\sigma_j \approx 344$
- at the cross-validated λ , $n\lambda = 10$, and 59 of the 83 directions are shrunk by more than half

Bonus: Ridge caps the variance

Under $y = X\theta + \varepsilon$, with σ^2 the noise variance:

- $\mathbb{E}\hat{\theta}_\lambda = \sum_j \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} \langle v_j, \theta \rangle v_j$: biased toward 0, most in the directions with small σ_j

Bonus: Ridge caps the variance

Under $y = X\theta + \varepsilon$, with σ^2 the noise variance:

- $\mathbb{E}\hat{\theta}_\lambda = \sum_j \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} \langle v_j, \theta \rangle v_j$: biased toward 0, most in the directions with small σ_j
- $\text{Var} \langle v_j, \hat{\theta}_\lambda \rangle = \sigma^2 \frac{\sigma_j^2}{(\sigma_j^2 + n\lambda)^2} \leq \frac{\sigma^2}{4n\lambda}$ (why?), against σ^2/σ_j^2 for least squares

Bonus: Ridge caps the variance

Under $y = X\theta + \varepsilon$, with σ^2 the noise variance:

- $\mathbb{E}\hat{\theta}_\lambda = \sum_j \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} \langle v_j, \theta \rangle v_j$: biased toward 0, most in the directions with small σ_j
- $\text{Var} \langle v_j, \hat{\theta}_\lambda \rangle = \sigma^2 \frac{\sigma_j^2}{(\sigma_j^2 + n\lambda)^2} \leq \frac{\sigma^2}{4n\lambda}$ (why?), against σ^2/σ_j^2 for least squares
- advertising: $1/\sigma_{\min}^2 = 1.2 \times 10^5$, while $1/(4n\lambda) = 0.025$

Bonus: Ridge caps the variance

Under $y = X\theta + \varepsilon$, with σ^2 the noise variance:

- $\mathbb{E}\hat{\theta}_\lambda = \sum_j \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} \langle v_j, \theta \rangle v_j$: biased toward 0, most in the directions with small σ_j
- $\text{Var} \langle v_j, \hat{\theta}_\lambda \rangle = \sigma^2 \frac{\sigma_j^2}{(\sigma_j^2 + n\lambda)^2} \leq \frac{\sigma^2}{4n\lambda}$ (why?), against σ^2/σ_j^2 for least squares
- advertising: $1/\sigma_{\min}^2 = 1.2 \times 10^5$, while $1/(4n\lambda) = 0.025$

Bonus: Ridge caps the variance

Under $y = X\theta + \varepsilon$, with σ^2 the noise variance:

- $\mathbb{E}\hat{\theta}_\lambda = \sum_j \frac{\sigma_j^2}{\sigma_j^2 + n\lambda} \langle v_j, \theta \rangle v_j$: biased toward 0, most in the directions with small σ_j
- $\text{Var} \langle v_j, \hat{\theta}_\lambda \rangle = \sigma^2 \frac{\sigma_j^2}{(\sigma_j^2 + n\lambda)^2} \leq \frac{\sigma^2}{4n\lambda}$ (why?), against σ^2/σ_j^2 for least squares
- advertising: $1/\sigma_{\min}^2 = 1.2 \times 10^5$, while $1/(4n\lambda) = 0.025$

Key idea: Least squares' variance in direction v_j grows without bound as $\sigma_j \rightarrow 0$; ridge's is at most $\sigma^2/(4n\lambda)$ in every direction.